SWE5006 PRACTICE PROJECT · TEAM 8

BLOCBUY

ONE FOR ALL • ALL FOR ONE

A scalable community group-buying platform

Two prices • pay first to team up • refunded if the team doesn't fill

Lim Jiechao · Liu Jin · Panpakorn Siripanich · Wang Ziang · Yu Qianshuo

Graduate Certificate in Designing Modern Software Systems · 26 October 2026

Group buying only works if enough people join in time

A market coordination failure, solved with a pay-first team purchase modelled on the early Pinduoduo playbook (proposal §4)

Today

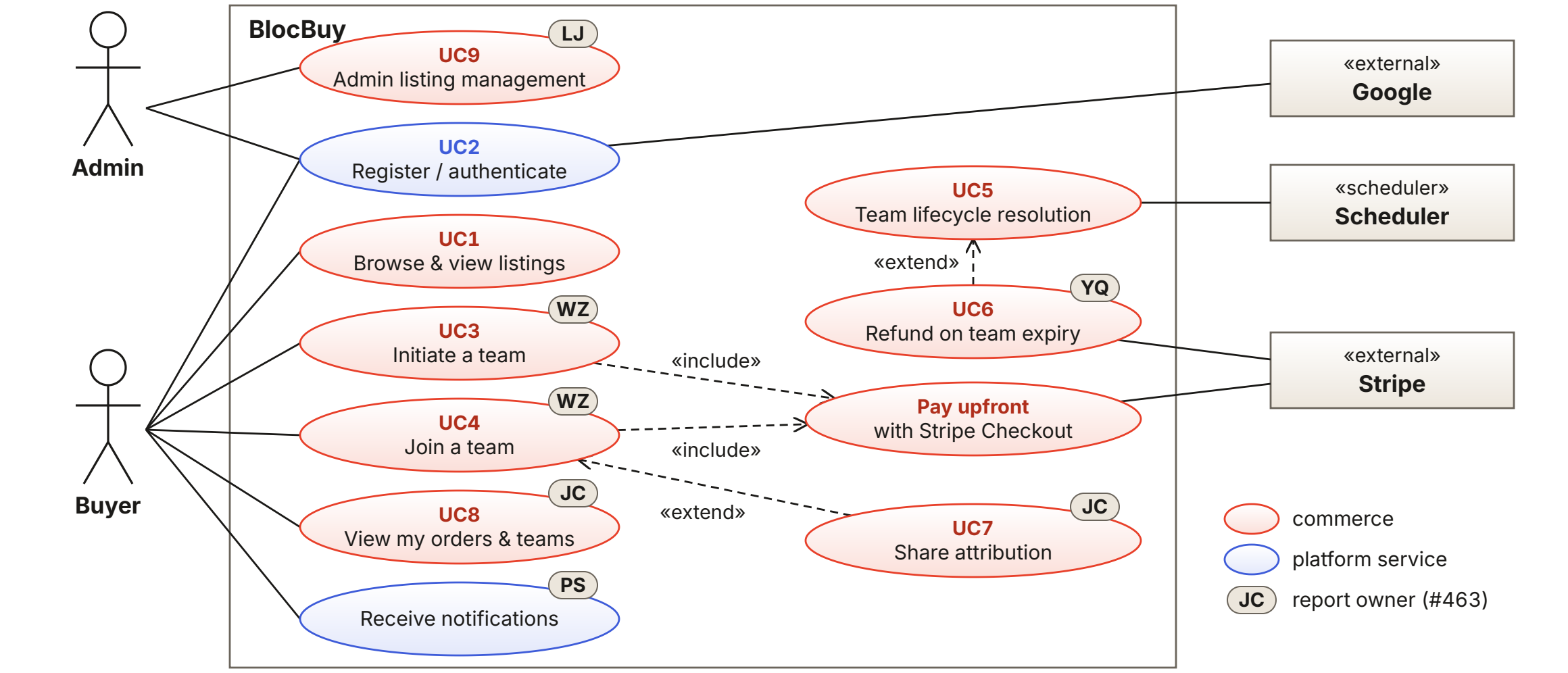
- Merchants can discount only when volume is predictable
- One buyer can never reach a bulk price
- Advertising is costly and not tied to real purchases

With BlocBuy: pay-first teams

- Two prices: individual and team
- Pay upfront to start a team, then share the invite link
- Friends join by paying upfront too
- Fills in time: team price. Expires: everyone refunded automatically

Stripe test mode only: no live money moves

Scope: nine use cases on one platform



Out of scope: merchant self-service GUI · analytics · recommendations · automatic team matching · email, SMS or push · native apps · live payments

JC Lim

Four sprints, one board, one Definition of Done

Sprint 1 — 2–12 Sep

Walking skeleton: CI/CD and dev live on 5 Sep; sign-in, browse, admin

Pipeline live; features late

35 team PRs · 3 issues closed

Sprint 2 — 14–25 Sep

One idempotent chain: payment → team → order

9 of 11 stories · 134 of 150 h

64 team PRs · 47 issues closed

Sprint 3 — 28 Sep–9 Oct

Invite → pay → refund on the deployed dev site

Not met on dev · 117 of 150 h

126 team PRs · 67 issues closed

Sprint 4 — 12–23 Oct

Complete system on dev; three services on ECS for SWE5001

`[FILL: result, 23 Oct]`

38 team PRs · 17 closed so far

Planning — plan.md, hours

Day log — 35 dated logs

Mid-sprint — check, S3–S4

Review — review, report

Retro — in the report

159 cards on the board: 132 Done, 7 in progress, 20 to do

356 PRs merged: 263 team, 70 Dependabot, 23 snapshot

Definition of Done, Sprints 3–4 (highlights)

- Acceptance criteria met; contract before controller
- Every gate green: Maven verify, ArchUnit, Gitleaks, Trivy, SonarCloud, `pnpm check`
- **"On dev" means deployed**: a local run does not count
- PR under ~1,500 lines; remaining hours on the board daily

Was it met?

- Required gates block every merge
- Sprint 3 not counted: Checkout off on dev
- Daily remaining hours not kept

Milestones and effort per member

27 Aug	5 Sep	14 Sep	22 Sep	27 Sep	8 Oct	11 Oct	15 Oct	23 Oct
Proposal	Backend live on dev	Sprint 1 tag	Web app live on dev	Sprint 2 tag	Startable listings on dev	Sprint 3 tag	Three services merged	Sprint 4 tag [FILL: date]

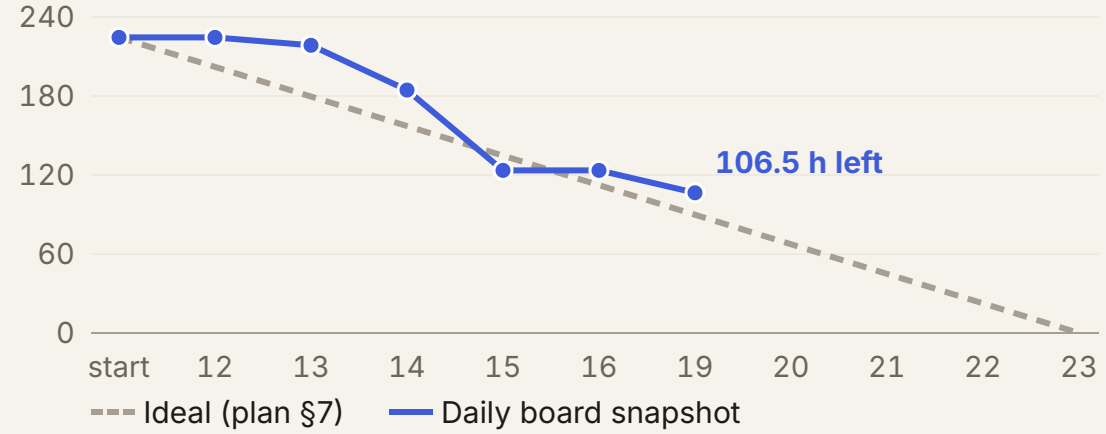
Planned hours per member [VERIFY: S4 split, PR #538]

Member	S2	S3	S4	Board items Done
Liu Jin	30	30	51	39
Yu Qianshuo	30	30	48	8
Wang Ziang	30	30	46	17
Panpakorn	30	30	41.5	29
Lim Jiechao	30	30	38	29

S1 hours were not recorded. Board items are delivery evidence, not effort. Planned S2–S4: 98–111 h each; guideline ≈ 10 man-days each (proposal §7)

[CONFIRM: actual effort per member, agreed by all five]

Sprint 4 burndown to 19 Oct: plan hours not Done



S1		[FILL: S1 hours]
S2		**134** of 150 h
S3		**117** of 150 h
S4		**118** of 224.5 h [FILL: final]

JC Lim

Status, open issues, and what each retro changed

Live on dev · 19 Oct

`[VERIFY: re-check on 25 Oct]`

- API health UP; web app loads
  checked 19 Oct, 14:14 SGT
- 14 listings in the catalog
  startable listings set through the admin API (8 Oct)
- Google sign-in, Secure session cookie
  NFR-01 verified on dev, 8 Oct
- Stripe Checkout opens and returns
  Sandbox, 16 Oct (#517)

Still open

- Webhook never confirms: payments stay “waiting”
  #517 · Yorik
- Scheduling and expiry refunds off on dev
  #223 · Yorik · go/no-go for the live payment demo
- Payment hardening in progress
  #539, #548, #550 · Eric
- Sprint 3 review and retro not yet written
  #451 · Lim

Retro → next sprint

Sprint 1 → 2
No burndown kept → daily board snapshots, hours per story

Sprint 2 → 3
A 10,000-line PR, cliff burndown, owners drifted → PRs under 1,500 lines, mid-sprint check, hours daily

Sprint 3 → 4
`[FILL: retro, #451]` → review: goal not met on dev, so #223 became the critical path

Sprint 4
`[FILL: retro on 23 Oct]`

WZ Eric

Top-down: split by transaction, one codebase, two topologies

One codebase: five COLA modules

modular monolith

`blocbuy-adapter`	REST controllers, Result envelope
`blocbuy-application`	use-case services, subscribers
`blocbuy-domain`	aggregates, events, ports; no Spring
`blocbuy-infrastructure`	MyBatis, Stripe, Redis, SQS, SNS
`blocbuy-client`	DTOs, integration event contracts

Each bounded context is a package in every module

identity · catalog · groupbuy · order · payment · notification

18 ArchUnit rules fail the build on a wrong dependency

Two topologies from one build

ADR 0008

DEV · SWE5006 · ONE EC2 HOST, DOCKER COMPOSE

Monolith: all six contexts in one process
events in-process, no broker

PROD · SWE5001 · AMAZON ECS

identity-svc

commerce-svc
catalog · groupbuy · order · payment

notification-svc

Events: outbox → SNS → SQS (ADR 0007)

Split only where no transaction crosses: pay, join, order and stock commit together, so they stay in commerce

Analysis → design: our transition rule

use cases: slide 3 · worked examples: B7

Bounded context	Analysis entity	External actor (Google, Stripe)	Use case
→ package in each module	→ aggregate + repository port	→ gateway port + adapter (ACL)	→ application service + endpoint

LJ Liu Jin

UC9 listings: State, Optimistic Offline Lock, and the schema

USE CASE · ANALYSIS

UC9 · An admin drafts, publishes and delists listings and sets team terms

ANALYSIS → DESIGN

listing → `Product` «aggregate»
listing status → `ProductStatus` (State)
listing store → `ProductRepository` port

DESIGN PROBLEM

Legal moves depend on the status, and two admins can edit the same listing

State GoF · behavioural

`ProductStatus.java`

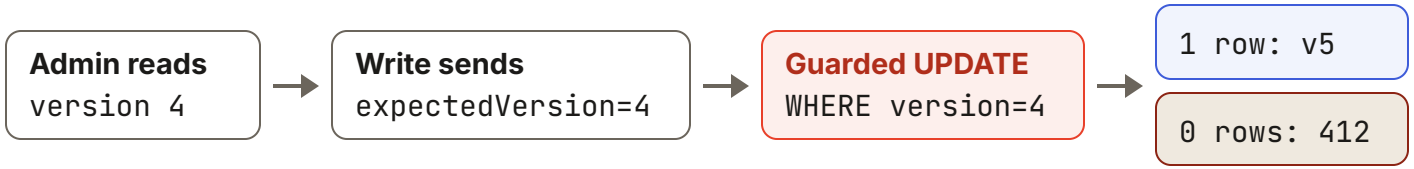


only state that may be deleted

Each constant overrides only its legal moves; every other move is refused with **409**

Optimistic Offline Lock Fowler, PoEAA

`Product.requireVersion`



A stale write changes nothing: **412**, and the console offers Reload (frontend ADR 0015)

Database schema

29 tables · 28 Flyway migrations

identity	`sys_user, sys_role +4`
catalog	`biz_product +3`
groupbuy	`biz_group_deal +4`
order	`biz_order, biz_purchase_intent`
payment	`biz_payment, biz_refund +4`
notification	`biz_notification, sys_consumed_message`
jobs	`sys_outbox +3`

UUID keys, timestamptz, numeric money; UNIQUE keys make retries safe · full list B5

WZ Eric

UC3/UC4 start and join: Factory Method and shared guards

USE CASE · ANALYSIS

UC3/UC4 · A buyer pays first to start a team, or pays to join an open one

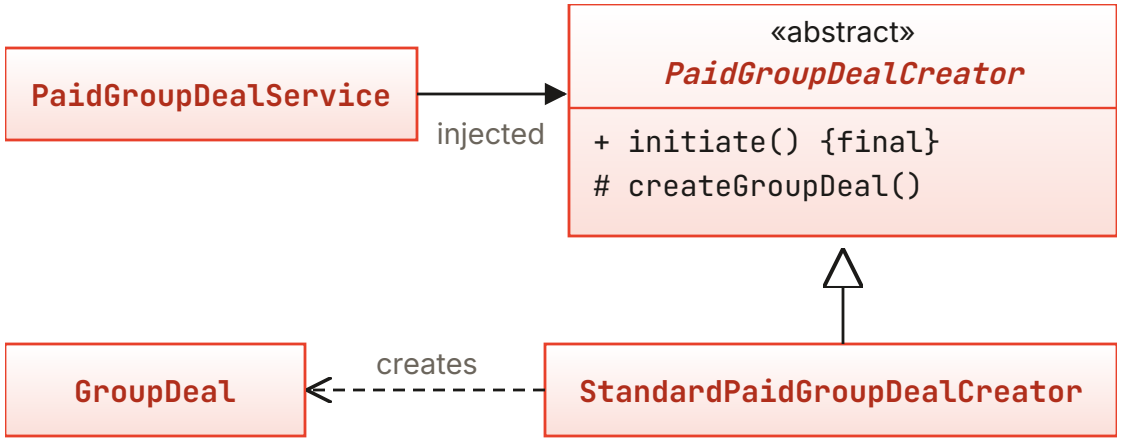
ANALYSIS → DESIGN

team → `GroupDeal` «aggregate»
accepted join → `PaidAdmission`
team store → `PaidGroupDealRepository` port

DESIGN PROBLEM

Join rules were written twice, for the quote and after payment, and drifted: a buyer could pay, then be refunded

Factory Method GoF · creational



`initiate()` is final: it checks terms and price, then calls `createGroupDeal()`, so it is also a Template Method

One creator, wired at build time by `GroupDealCreationConfig` (#421): a seam, not runtime-pluggable

Shared guards on the aggregate domain methods, not GoF

BEFORE PR #424

Quote
`lockJoinTerms`
own inline checks

After payment
`joinPaid`
own inline checks

Team with 0 members: quote OK, DEAL_FULL after payment, refund

AFTER (#422, MERGED 13 OCT)

`lockJoinTerms`
`joinPaid`
→ **GroupDeal**
`requireJoinWindow()`
`requireAvailableSeat()`
`requireNewMember()`

Each path keeps its own lock and check order

YQ Yorik

UC6 expiry refund: never refund twice

USE CASE · ANALYSIS

UC6 · When a team expires, every paid member is refunded automatically

ANALYSIS → DESIGN

refund owed → `biz_refund` + `RefundWork`

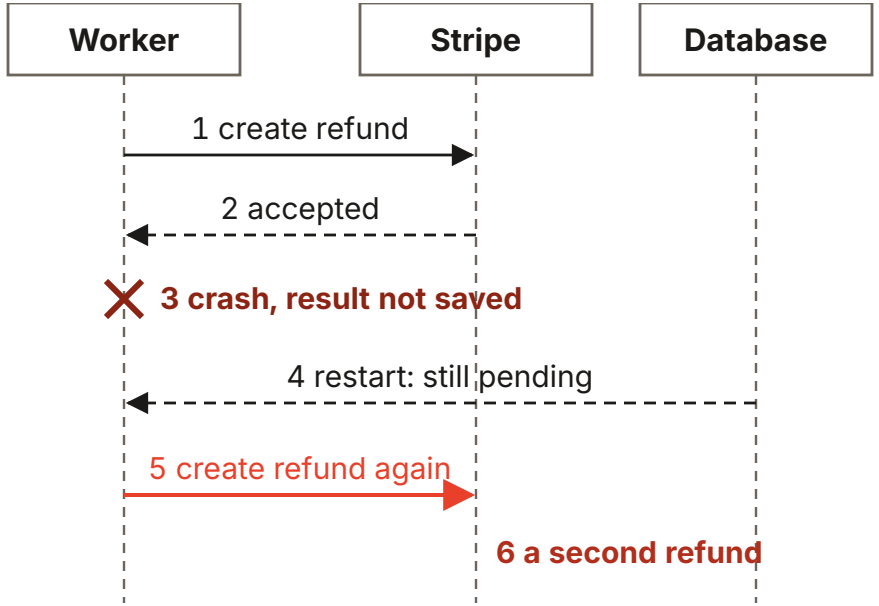
payment provider → `RefundGateway` port

Stripe → `StripeRefundGateway` adapter

DESIGN PROBLEM

Stripe accepts a refund and the process dies before saving it: a naive retry refunds twice

THE CRASH WINDOW



Without a stable request identity, the retry is a new refund

SOLUTION

Idempotency Key	`RefundWork.providerKey()` = "blocbuy-refund-" + id, sent by `StripeRefundGateway`: a retry gets the same refund back
Lease + version	A claim leases the row for 2 min and bumps its version; the result is saved only if the version still matches
Fixed retry	A failed call is due again after a fixed 30 s, not exponential backoff
Human review	Payment mismatch, permanent provider error, or no result after 23 h: parked for review, never a new refund
Observer	`TeamExpired` → order and refund subscribers, before the same commit (ADR 0002)

Proven locally with a Stripe substitute (3 Oct); no refund on dev yet (#223)

PS Pan

Notifications: Observer, Outbox, Idempotent Receiver

USE CASE · ANALYSIS

Receive notifications · A buyer is told when their team succeeds or expires, or a payment succeeds

ANALYSIS → DESIGN

notice → `Notification` «aggregate»
commerce fact → `TeamExpiredV1` contract
queue message → `NotificationEventHandler` port

DESIGN PROBLEM

Tell buyers without commerce calling notification, and never lose or repeat a notice

OBSERVER	TRANSACTIONAL OUTBOX	RELAY	TRANSPORT	IDEMPOTENT RECEIVER
domain event `TeamExpired` published in-process	**BEFORE_COMMIT** `IntegrationOutboxWriter` same transaction → sys_outbox	**every 2 s, leased** `OutboxRelayJob` batches of 100, 10 tries	**at least once** `SNS → SQS` DLQ after 5 receives	**one transaction** `markConsumed` then biz_notification rows

Commerce never calls notification: the ArchUnit rule `nothing_depends_on_notification` fails the build

IDEMPOTENT RECEIVER IN SQL

```
INSERT INTO sys_consumed_message
  (event_id, consumer, …)
ON CONFLICT (event_id, consumer) DO NOTHING
```

0 rows inserted = a duplicate: nothing written, message deleted

WHERE IT RUNS

dev · EC2	monolith: writer, relay and consumer off; no broker (ADR 0007)
prod · ECS	commerce-svc → SNS → SQS → notification-svc
local, CI	SQS on ElasticMQ: OutboxRelayElasticMqIT, NotificationConsumerElasticMqIT

JC Lim

UC8 My blocs: the URL owns the view state

USE CASE · ANALYSIS

UC8 · A buyer sees their blocs (Active, Completed, Refunded) and orders on /account

ANALYSIS → DESIGN

My blocs → `GET /group-deals/me`, latest 100

My orders → `GET /orders/me`, paged

view state → URL `?blocs=&page=&size=`

DESIGN PROBLEM

A filter and a pager must survive reload, Back and a shared link; the API has no status filter `[CONFIRM: Lim picks the problem]`

URL as the single source of view state

router owns URL state, Query owns server state (frontend ADR 0001)

URL
`/account?blocs=completed&page=2`

→

validateSearch
`accountSearchSchema`
bad value → fallback; defaults stripped

→

loader
`orders + blocs`
prefetched in parallel; past the last page → redirect

→

chip click
`?blocs=`
not a loader dependency: filters the cache, no refetch

Adapter at one boundary

frontend ADR 0009

generated client
`Result<T>`

→

select:
`unwrapResult`

→

components
`T or ApiError`

One function knows the envelope; the client is generated from the contract (ADR 0003)

EVIDENCE · ACCOUNT.TEST.TSX

- switches chips through ?blocs= without refetching
- falls back to Active for an unknown ?blocs= value
- sends a page past the last one to the last page

Limit: only the latest 100 blocs; a closed bloc reads “Closed”, as this endpoint carries no refund data

LJ Liu Jin

Seven kinds of tests, from domain rules to the deployed site

KIND OF TEST	COUNT	TOOL · WHEN IT RUNS	EXAMPLE: COMPLEX BEHAVIOUR UNDER TEST (3B)
Performance	**3** load scenarios	Node workload · k6 · by hand	`backend/perf/workload.mjs` 100 users each, p95 24–57 ms (local, 30 Sep)
End-to-end	**7** specs · 18 tests	Playwright · every PR · after deploy	`buyer-journey.spec.ts` listing → quote → Stripe (stubbed) → Paid in My blocs
Component	**76** files · 799 tests	Vitest · Testing Library · MSW · every PR	`account.test.tsx` a chip switches ?blocs= without refetching
API contract	**1** OpenAPI file	export · git diff · openapi-diff · every PR	`contracts/openapi.json` drift or a breaking change fails the build
Architecture	**18** ArchUnit rules	ArchUnit · every PR	`ArchitectureTest` domain_is_framework_free, nothing_depends_on_notification
Integration	**49** classes	Spring Boot · Testcontainers · every PR	`OrderRepositoryIT · StripeRolloutAcceptanceIT` PostgreSQL 18 rejects a duplicate payment; paid start to refund
Unit	**182** classes	JUnit Jupiter · Mockito · every PR	`RefundServiceTest · PaidGroupDealTest` a lost Stripe response reuses its key; 24 paid-team rules

backend frontend

Counted in the repository, 19 Oct · security scans (gitleaks, Trivy): Pan's segment

LJ Liu Jin

Gates that block a merge, and what tests caught

Required to merge into dev

ruleset, strict

- build-quality: every test
- docker: image boots
- gitleaks
- SonarCloud Backend
- SonarCloud Frontend
- frontend-check: Vitest

Branch up to date · 1 approval · threads resolved · squash only

SonarCloud gate on dev, 19 Oct

NFR-06

Fails under **80%** coverage or over **3%** duplication on new code

BACKEND

99.2% on new code

99.4% overall · JaCoCo, 16 Oct

FRONTEND

96.2% on new code

96.1% overall · Vitest V8, 19 Oct

1,931 backend tests

0 failed, 1 opt-in skip · log 16 Oct

799 frontend tests

e2e 17 passed, 5 skipped · 19 Oct

WHAT TESTS CAUGHT BEFORE PRODUCTION



UNIT · RED FIRST

A team with no members passed the quote

The buyer could pay, then be refunded. Two red tests reproduced it before the fix (slide 9).

GroupDealPurchaseSupportImplTest · 7 Oct · #422



ARCHITECTURE

A new controller skipped input validation

`controllers_are_validated` failed the build until it had `@Validated`.

ArchitectureTest · 13 Oct · #468



END-TO-END AFTER DEPLOY

Dev answered 502 during a deploy

The smoke test ran while the single backend host restarted: dev deploys are not zero-downtime.

e2e-dev · 3 and 4 Oct · runs 37102432667, 37182996807

Limits: *test-first only for selected changes · no automated mutation testing (hand probes only) · load tests by hand, not in CI · CodeQL wired but off (ENABLE_CODEQL=false) · contract, e2e and Trivy run on PRs but are not required*

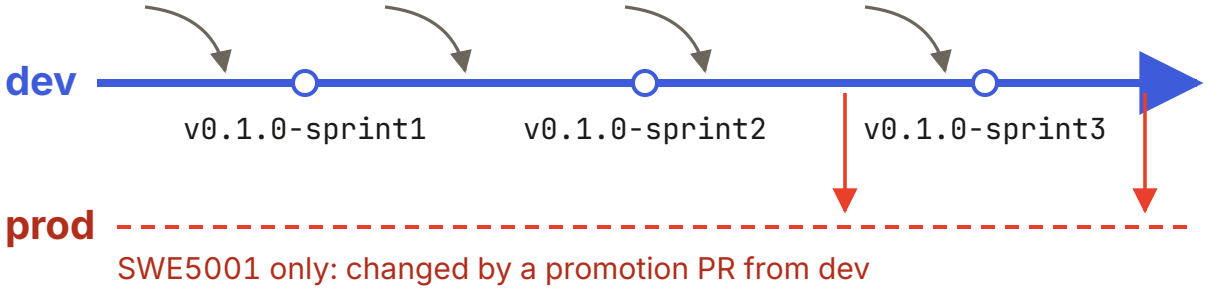
PS Pan

Source code and artefacts: one trunk, images by digest

Source: one trunk, squash only

ruleset dev-require-review

feature/*, fix/*, docs/* · one pull request each



SWE5001 only: changed by a promotion PR from dev

- Squash merge only: linear history
- 1 approval by a code owner; stale approvals dismissed
- Review threads resolved
- 6 required checks, branch up to date
- No force push, no branch deletion
- Conventional Commits; PR template asks for evidence

CODEOWNERS .github/ → Pan · backend/ → Liu Jin · frontend/ → Lim

Dependabot: Maven, npm, Docker, Actions · daily, 7-day cooldown · 70 PRs merged

Artefacts

Backend image in ECR `dev/blocbuy`
Immutable tags, scanned on push, newest 30 kept; deployed as `repo@sha256:…`, never by tag

Frontend `dist/`
One static build, no per-environment settings (ADR 0004); hashed assets immutable, `index.html` no-cache

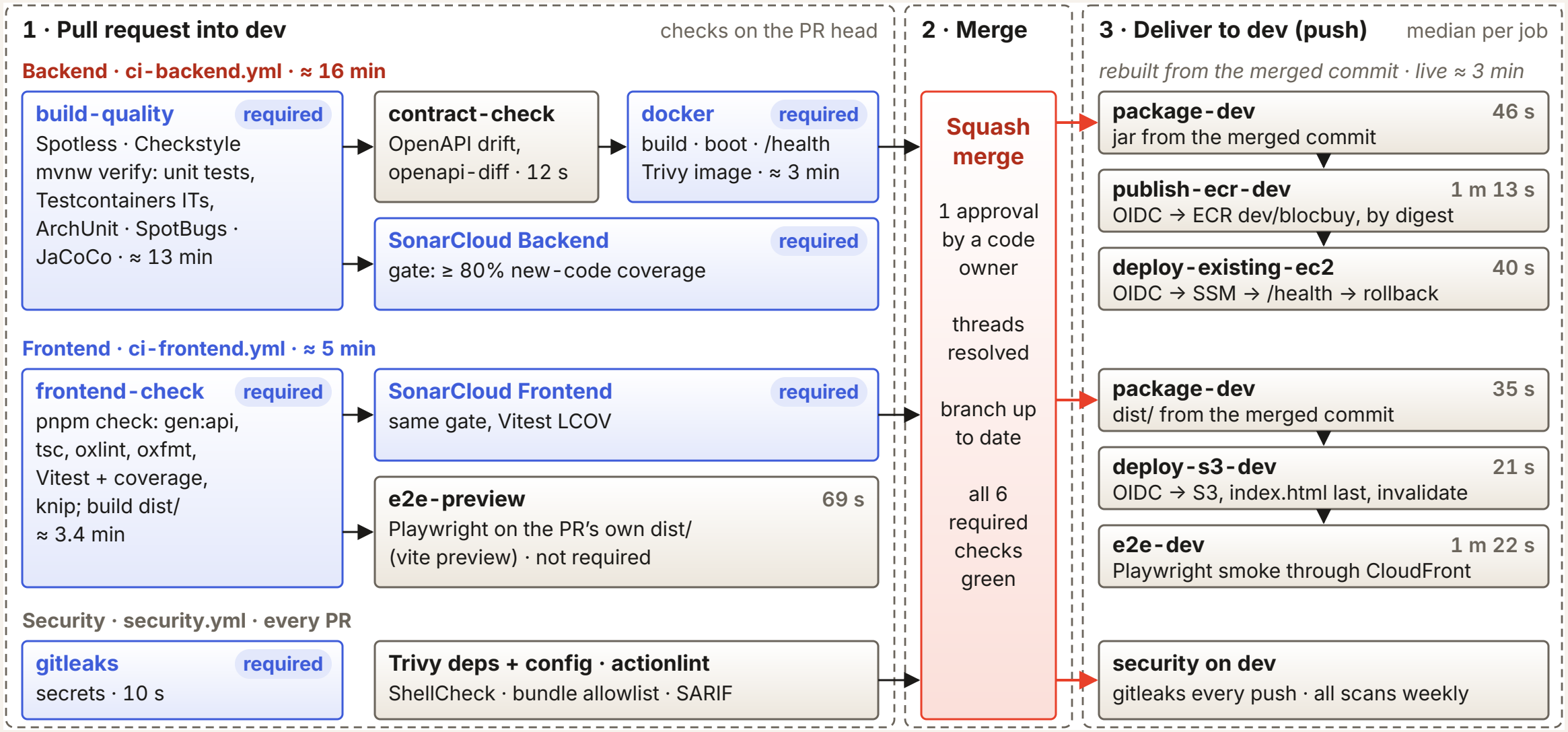
API contract
`contracts/openapi.json` regenerated from code; drift fails CI; the client is generated, not committed (ADR 0003)

Release tags
`v0.1.0-sprint1…3`; a tag runs the full gate chain

After merge, the jar, image and `dist/` are rebuilt from the merged commit, then deployed (slide 16)

PS Pan

One pipeline from pull request to dev



PS Pan

Security in the pipeline, no stored keys

Shift-left controls

security.yml · ci-backend.yml

Control	Catches	Runs	Gate
gitleaks	secrets in commits	PR + dev push	required
SonarCloud	bugs, hotspots, coverage	PR + dev	required
SpotBugs, ArchUnit	Java bugs, layering	every PR	required
Trivy image	fixable HIGH/CRITICAL	PR; ECR on push	required
Trivy deps, config	Maven, pnpm, IaC	on change, weekly	fails run
actionlint, ShellCheck	workflows, scripts	on change	fails run
Pinned actions	all 28 on commit SHAs	Dependabot daily	review

Trivy writes SARIF reports; path-filtered scans also run on tags and every Monday

No AWS keys in GitHub

OIDC

`BlocBuyGitHubEcrPush`
trust: dev branch · push to dev/blocbuy

`BlocBuyGitHubBackendDeploy`
trust: backend-dev env · one SSM document, one instance

`BlocBuyGitHubFrontendDeployDev`
trust: frontend-dev env · one bucket, one distribution

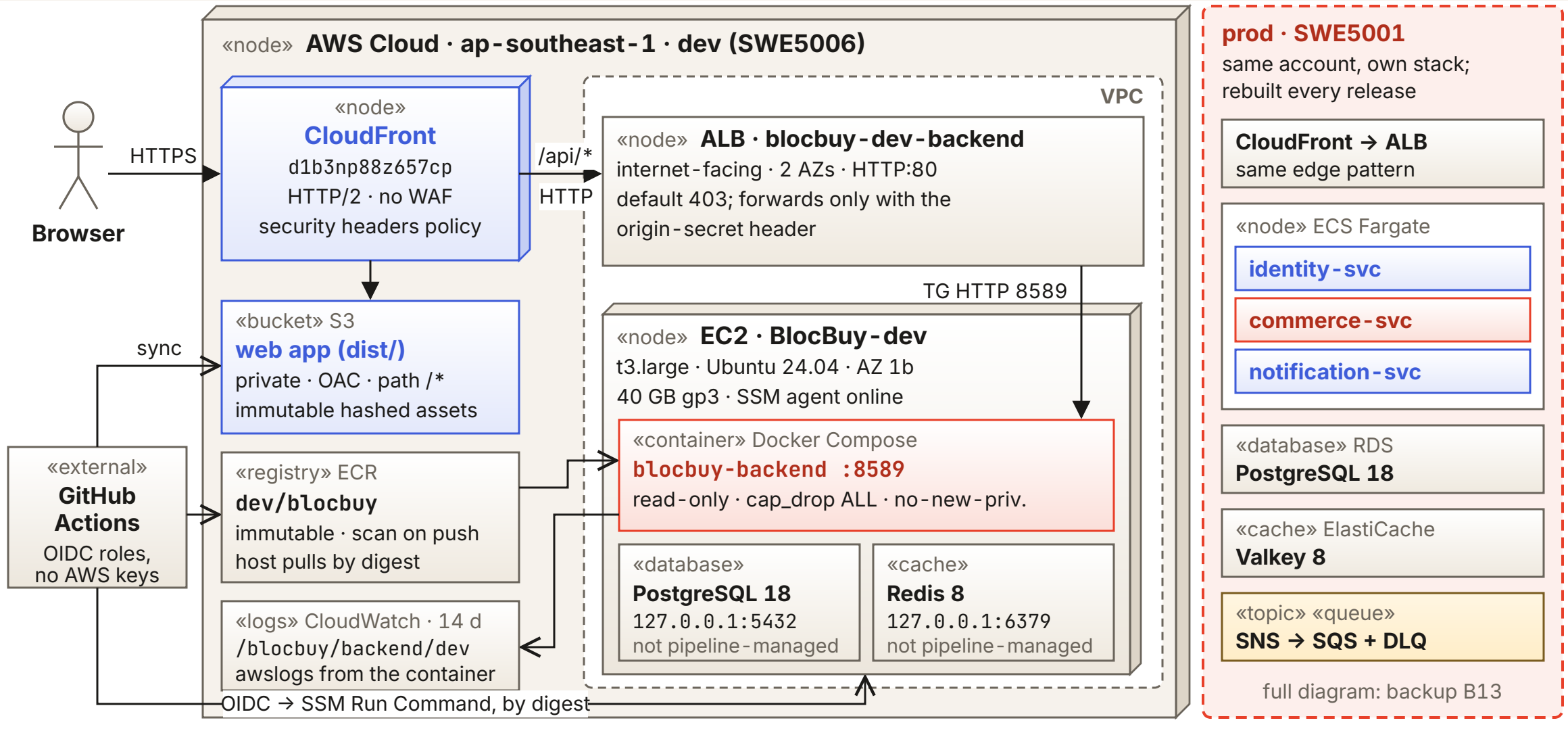
Runtime and account

- Container: read-only root, `cap_drop: ALL`, no-new-privileges
- App secrets only in a root-only file on the host (0600)
- AWS Organization SCPs pin the region and cap costs; Identity Center, 1-hour sessions

Not yet: *CodeQL wired but off (ENABLE_CODEQL=false) · CloudFront → ALB hop is HTTP (NFR-37, #227) · dev host: SSH open to the internet, EBS not encrypted, secrets in an env file · full list B14*

PS Pan

Where it runs: dev on EC2, prod on ECS



PS Pan

Technical findings and issues

DEV · PIPELINE GREEN, FEATURE NOT

Payments never confirm on dev

Stripe Checkout opens and returns, but the webhook never moves the payment out of “waiting”. Deploys and health checks stay green.

Open: find the endpoint, secret or event mismatch; go/no-go for the payment demo on 23 Oct

#517, #223 · 16 Oct · Yorik

DEV · SINGLE HOST

A deploy answered 502

The e2e-dev smoke hit the API while the only backend container was being replaced.

Accepted: dev deploys are not zero-downtime: one host, container replaced in place

runs 37102432667, 37182996807 · 3–4 Oct

DEV · OPERATIONS

Guardrails also block operators

Organisation SCPs denied Access Analyzer and bucket-list views; scoped deploys still worked.

Fix: IAM changes through the CLI; four SCPs consolidated into three (#530)

day log 23 Sep · log observability guide · 16 Oct

SWE5001 · REAL CLOUD ≠ MOCKS

Tests passed, the real run failed

Terraform tests on mocks passed, then AWS refused a tag permission; a read-only root on Fargate left Tomcat no writable /tmp.

Fix: scoped TagResource grant (#527); `VOLUME /tmp` in both Dockerfiles

PR #527 · 16 Oct · day log 6 Oct

PS Pan

Live: one change from pull request to dev

1. Open PR
a small frontend change

→

2. Checks
required checks, e2e-preview

→

3. Approve
code owner

→

4. Squash merge
into dev

→

5. Deliver
S3, CloudFront, smoke test

→

6. See it
on the dev site

≈ 5 min
frontend PR checks (median 4.8 min, 19 runs, 16–19 Oct)

≈ 2.7 min
merge → live and smoke-tested (median, 94 runs)

≈ 3.1 min
backend merge → deployed (median, 88 runs); shown recorded

Recorded fallback ready
If GitHub or AWS is slow, we switch to the recording (item 4e)
`[FILL: link, recorded by 25 Oct]`

Watch for
the required checks turning green, the merge, then the new version on `d1b3np88z657cp.cloudfront.net` `[CONFIRM: the demo change]`

WZ Eric

Live demo: one bloc from invite to refund

1

Start a bloc and share the invite

UC2 sign in · UC3 start · UC7 invite · Stripe Sandbox

LIVE ON DEV

Functionality, security

→

2

A friend joins through the invite and pays

UC4 join · UC7 credit to the sharer · Stripe Sandbox

LIVE ON DEV

Consistency

→

3

My blocs and my orders

UC8 · Active, Completed, Refunded

LIVE ON DEV

Functionality

→

4

The bloc started at 16:05 expires and refunds

UC5 resolve · UC6 refund · receipt

LIVE ON DEV

Reliability, consistency

→

5

Quality attributes

one refund per payment · ECS scale-out · load tests · the pipeline

LABELLED EVIDENCE

Scalability, performance, maintainability

Live on dev: one EC2 host, Stripe Sandbox test cards, no real money

Labelled: SWE5001 prod on ECS or a local run, named on screen

- **Live on dev; recorded steps are marked** `[CONFIRM: go/no-go for the live payment steps on 23 Oct (#223, #517)]`

Thank you

Questions?

WHAT WE HOPE YOU REMEMBER

- Pay first; refunded automatically if the team fails
- A refund is never paid twice, even after a crash
- One codebase, two topologies: EC2 and ECS
- Pull request to dev in minutes, every gate green

99.2%
backend coverage on new code (SonarCloud, 19 Oct)

1,931
backend tests, 0 failed (16 Oct)

≈ 2.7 min
frontend merge → live and smoke-tested (median)

github.com/PPsyrius/iss-groupbuy-team8 · d1b3np88z657cp.cloudfront.net

Backup slides: index

B1 **Context map**
How do the contexts relate, and where are the anti-corruption layers?

B2 **Aggregates and state machines**
Which aggregates hold the rules, and how does a team end?

B3 **UC4 join sequence**
What happens in one paid join?

B4 **Expiry and refund sequence**
How does an expired team become refunds?

B5 **Database schema**
Which tables does each context own?

B6 **UC6 candidates and decisions**
What else did you consider for refunds?

B7 **Transition from analysis to design**
How did analysis classes become design classes?

B8 **ADR index**
Which decisions are recorded, and where?

B9 **Logical view (ECS topology)**
How do the three services talk on ECS?

B10 **Test inventory**
How many tests of each kind, and where do they run?

B11 **Sprint 2 and 3 burndowns**
How did earlier sprints burn down?

B12 **Workflows, required checks and timings**
What runs when, how long, and how often is it green?

B13 **Production topology (SWE5001)**
What does prod look like on ECS?

B14 **Known gaps and limitations**
What is not done yet in the pipeline and on dev?

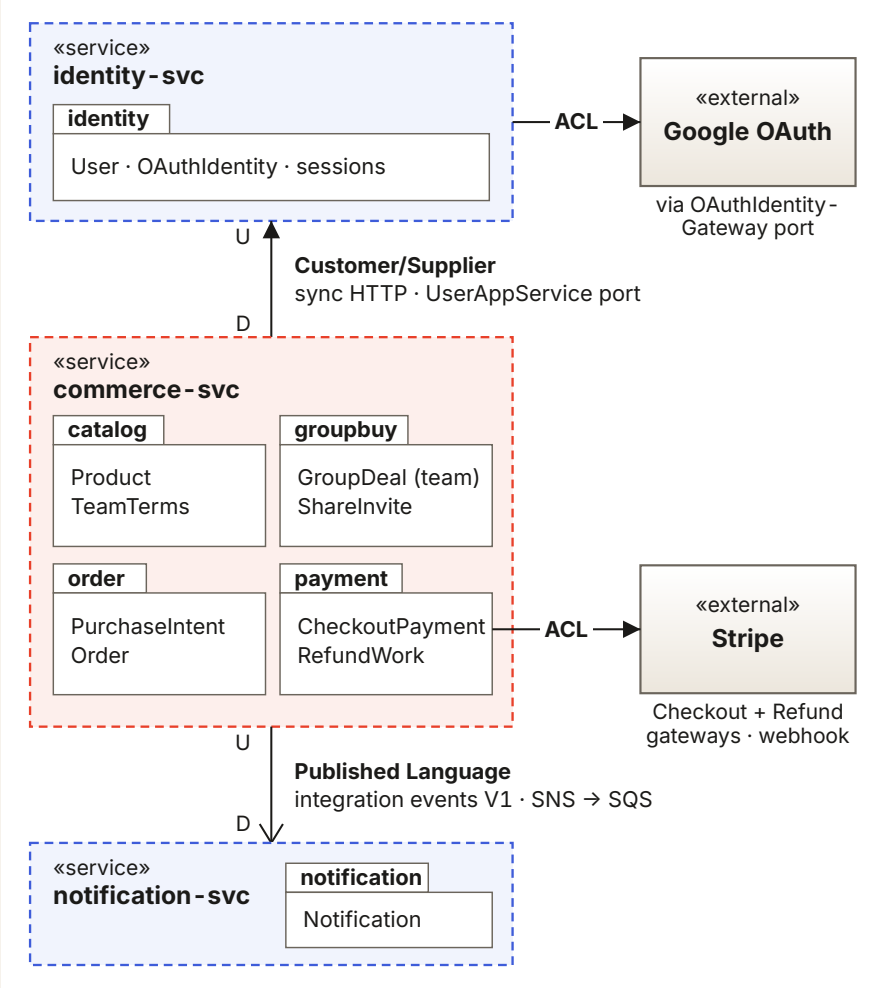
B15 **Stripe Sandbox refund, local evidence (20 Sep)**
Is the refund real, and where has it been proven?

B16 **Scale-out and load results (SWE5001 prod + dev baseline)**
How does it scale, and how fast is it?

Click a row (or type its number in the URL as #B6). Slide 22 stays up during Q&A; open a backup only when a question needs it.

WZ Eric

Context map



- **identity → Google:** anti-corruption layer, `OAuthIdentityGateway` port, `GoogleOAuthGateway` adapter
- **identity → commerce:** Customer/Supplier through the `UserAppService` port; on ECS the one synchronous HTTP call, on dev an in-process call
- **commerce → Stripe:** anti-corruption layer (Checkout and Refund gateways, signed webhook)
- **commerce → notification:** Published Language, `TeamSucceededV1`, `TeamExpiredV1`, `PaymentSucceededV1` over SNS → SQS

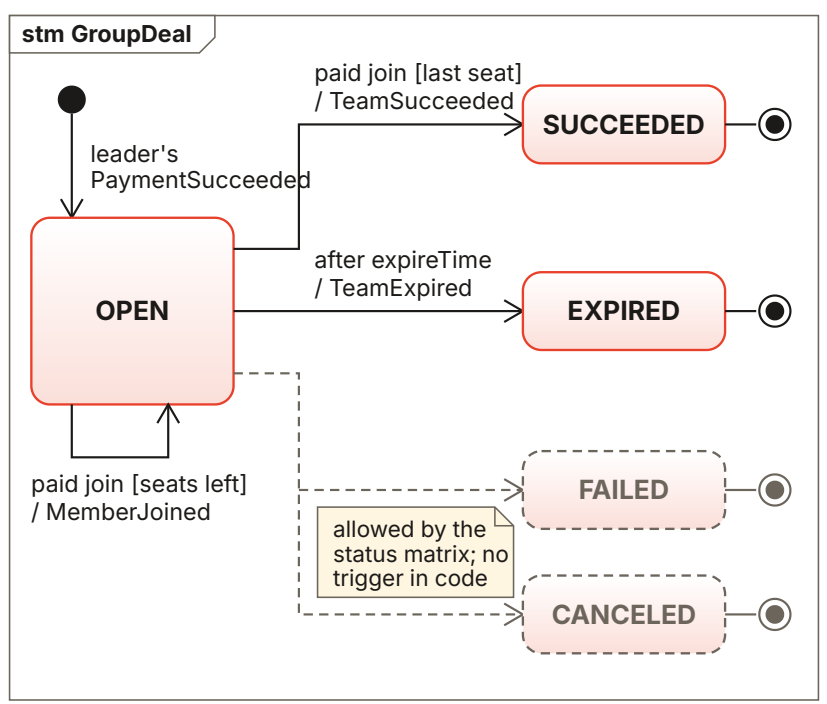
Drawn for the **ECS topology**. On dev the same contexts run in one monolith: the identity call is in-process, and no events leave commerce (no broker).

WZ Eric

Aggregates and state machines

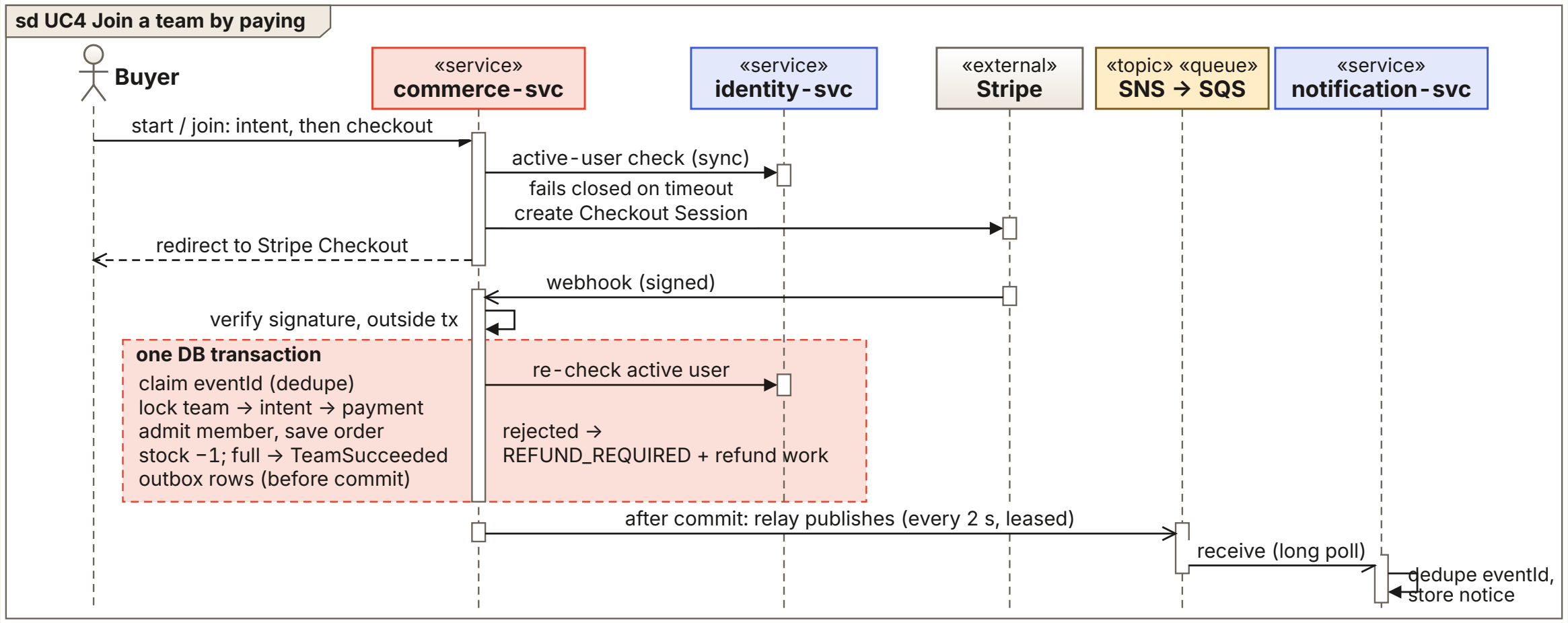
- **GroupDeal** (team): only OPEN may move, once; `DealStatus.canTransitionTo` is the one transition matrix (ADR 0002)
- **Order:** PENDING → CONFIRMED or CANCELLED
- **PurchaseIntent** is the order-side aggregate (ADR 0001)
- **Product** + `ProductStatus` (State, slide 8); also CheckoutPayment, RefundWork, Notification
- **Domain events:** MemberJoined, TeamSucceeded, TeamExpired, PaymentSucceeded; subscribers run before commit (ADR 0002), the share credit after commit (ADR 0003)

Be ready to explain: FAILED and CANCELED are allowed by the matrix but have no trigger in code. GroupDeal uses a matrix, not the State pattern.



WZ Eric

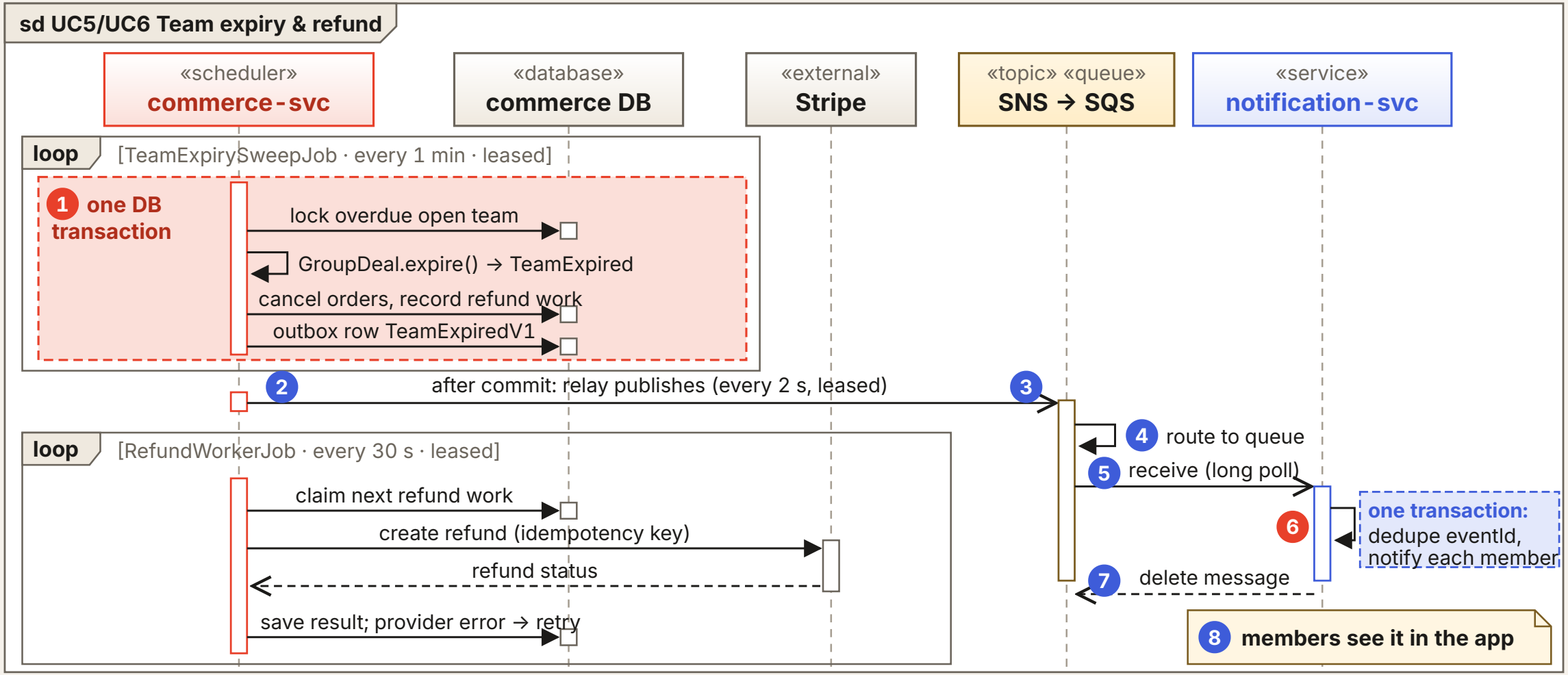
UC4 join sequence



As built for ECS. On dev the active-user check is in-process and there is no broker, so no SNS → SQS or notification steps.

YQ Yorik

Expiry and refund sequence



As built for ECS. The SNS → SQS hop and notification-svc (steps 2–8) are the ECS topology; on dev the monolith runs no broker.

LJ Liu Jin

Database schema

Context	Tables	Migrations	Keys that make “once” hold
identity	`sys_user, sys_user_oauth, sys_role, sys_permission, rel_user_role, rel_role_permission`	V3, V24	one binding per Google subject
catalog	`biz_product, biz_team_price_policy, biz_team_price_quote, log_product_team_terms`	V4, V6, V14, V20, V21	biz_product.version (Optimistic Offline Lock)
groupbuy	`biz_group_deal, biz_group_member, biz_group_participation_receipt, biz_share_invite, biz_share_credit`	V5, V7, V18	one member per user and team; one receipt per payment
order	`biz_order, biz_purchase_intent`	V12, V15, V19, V22	intent fulfilled by one payment
payment	`biz_payment, log_payment_webhook_event, biz_refund, log_payment_review, biz_payment_review_case, log_refund_attempt`	V2, V9–V11, V16	webhook event_id; one refund per payment
notification	`biz_notification, sys_consumed_message`	V27, V28	PK (event_id, consumer); (event_id, user_id)
jobs, outbox	`sys_outbox, sys_scheduler_lock, sys_scheduled_item_retry, sys_checkout_expiry_cursor`	V13, V17, V25, V26	one lease per job; outbox id = eventId

28 Flyway files, 29 tables in schema `domain`; V1 sets the rules: snake_case, timestamptz, numeric money, invariants in the database. On ECS (ADR 0008): one database per service on one RDS instance; identity-svc and notification-svc own their rows above, commerce-svc the rest.

YQ Yorik

UC6 candidates and implementation decisions

Candidate	Decision	Why
Mark refunded, then call Stripe	**Rejected**	A crash would report money returned that never left
Hold a DB transaction across the call	**Rejected**	A rollback cannot undo a refund; locks held over network I/O
Idempotency Key per obligation	**Selected**	Every retry reuses one request identity
Retry after a delay	**Selected**	Fixed 30 s in code, not exponential backoff
Human-in-the-loop review	**Selected**	Mismatched or ambiguous work leaves automatic claims
Distributed transaction with Stripe	**Rejected**	No shared protocol across PostgreSQL and the provider

Implementation decisions

1. Obligation commits with the expiry, before any I/O
2. Key from the obligation ID: `"blocbuy-refund-" + id`
3. Short claim and finish transactions; lease and version fence workers
4. Verify payment facts before, provider snapshot after
5. After 23 h: search Stripe by metadata, never create
6. Reconcile only retrieves an existing refund; no admin endpoint exists

Source: docs/swe5006/report-drafts/yorick-yu-contribution.md. Evidence is local with a provider substitute.

WZ Eric

Transition from analysis to design

Rule: bounded context → package in each module · analysis entity → aggregate + repository port · external actor → gateway port + adapter (anti-corruption layer) · use case → application service

Member · use case	Analysis class	Design class or port	Pattern
Liu Jin · UC9	Listing, its status	`Product`, `ProductStatus`, `ProductRepository` → `ProductRepositoryImpl`	State, Optimistic Offline Lock
Eric · UC3/UC4	Team, Membership, Purchase intent	`GroupDeal`, `PaidAdmission`, `PurchaseIntent`, `PaidGroupDealRepository`; payment settles through `PaymentBusinessPort` ← `OrderFulfilmentAdapter`	Factory Method
Yorik · UC6	Refund obligation, Payment provider	`biz_refund` + `RefundWork`, `RefundRepository`; `RefundGateway` ← `StripeRefundGateway`	Idempotency Key, Observer
Pan · notifications	Notice, commerce event	`Notification`, `NotificationRepository`; `TeamExpiredV1`; `NotificationEventHandler` ← `SqsNotificationEventConsumer`	Observer, Outbox, Idempotent Receiver
Lim · UC8	My blocs, My orders	`GET /group-deals/me`, `GET /orders/me` → route `/account`, `myTeamsQuery`, `accountSearchSchema`	URL as view state, Adapter
(identity)	Google account	`OAuthIdentityGateway` ← `GoogleOAuthGateway`	Adapter (ACL)

Sources: report drafts (Yorik, Eric's unmerged PR #542), backend design docs, code checked 19 Oct. ← means "implemented by".

WZ Eric

ADR index

Backend · backend/docs/adr

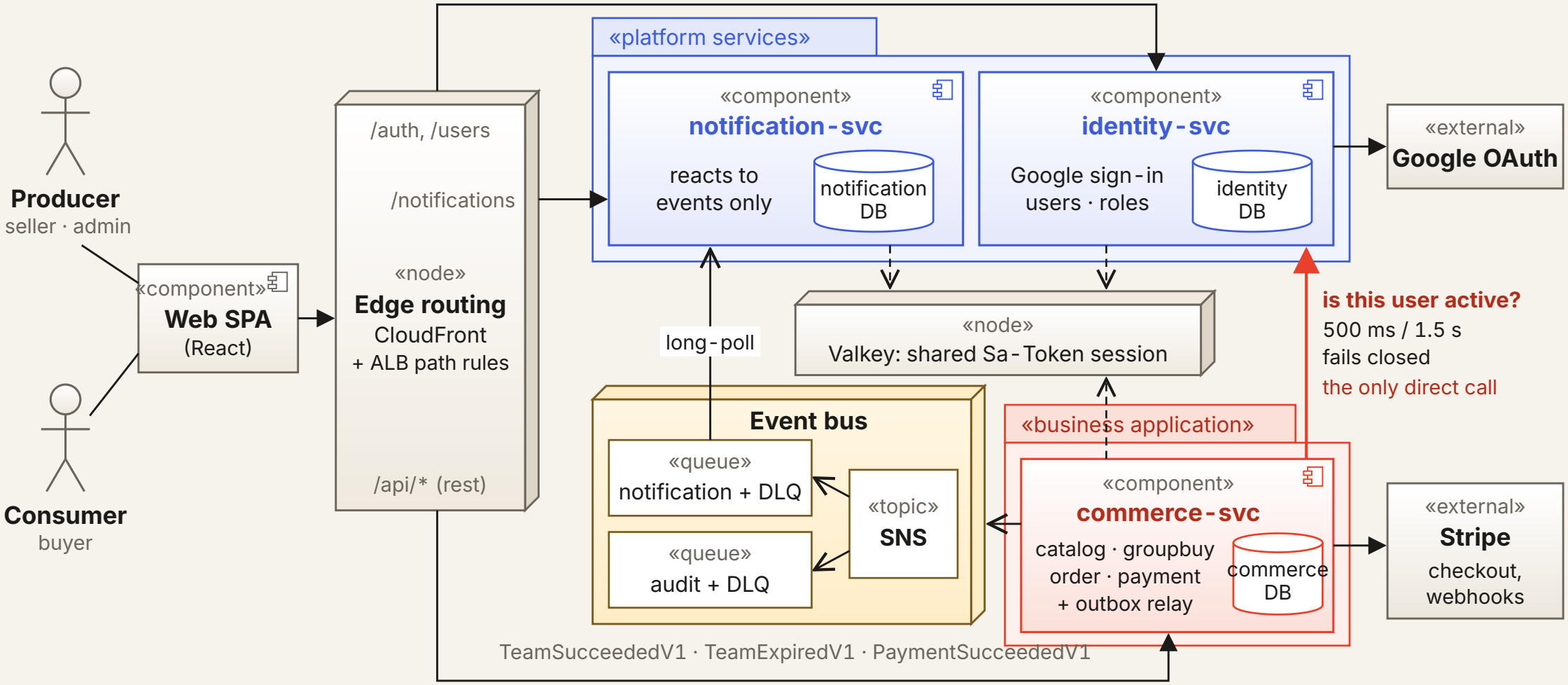
#	Decision
0001	Purchase intent lives in order; group terms on the product
0002	Team resolution is published; consequences run before commit
0003	Referral credit is a post-commit subscriber
0004	Retire the unused legacy PaymentIntent path
0005	Gate Stripe Checkout with a deployment property
0006	Payment owns the admin attention read model
0007	Outbox, one SNS topic, one SQS queue per consumer (Proposed)
0008	One codebase, two topologies: dev monolith, three services on ECS

Frontend · frontend/docs/adr

#	Decision
0001	React SPA, pure CSR; Router owns URL state, Query server state
0002	Same-origin /api with an HttpOnly cookie session
0003	Generated API client, not committed
0004	One build artifact promoted dev to prod
0005	SPA fallback by a CloudFront Function
0006	oxlint and oxfmt toolchain
0007	Hand-managed dev CloudFront, then Terraform import
0008	Documentation and evidence conventions
0009	Result envelope: one unwrapResult as select
0010	React Compiler via the Babel preset
0011	Testing strategy: Vitest, MSW, Playwright smoke
0012	Agent configuration in AGENTS.md
0013	pnpm and Node pinned, exact versions
0014	Stripe Hosted Checkout with a server-owned intent
0015	Admin console shell, roles guard, versioned writes

WZ Eric

Logical view (ECS topology)



SWE5001 prod on ECS: three services assembled from the same modules (backend ADR 0008). SWE5006 dev runs all of it as one monolith.

LJ Liu Jin

Test inventory

Kind	Tools	Count by module	Example classes	Runs
Unit	JUnit Jupiter, Mockito	182 classes: domain 42, application 41, infrastructure 47, bootstrap 43, adapter 4, platform 2, commerce-svc 2, client 1	`RefundServiceTest`, `PaidGroupDealTest`, `ProductStatusTest`	every PR (build-quality)
Integration	Testcontainers: PostgreSQL 18, Redis 8.10, ElasticMQ	49 classes: bootstrap 24, infrastructure 15, services 9, platform 1	`OrderRepositoryIT`, `ServiceGrantsIT`, `OutboxRelayElasticMqIT`, `StripeRolloutAcceptanceIT` (acceptance)	every PR (build-quality)
Architecture	ArchUnit	18 rules in 1 class	`ArchitectureTest`	every PR (build-quality)
API contract	springdoc export, git diff, openapi-diff 2.1.7	1 file	`contracts/openapi.json`	backend PRs (contract-check, not required)
Component	Vitest, Testing Library, MSW	76 files: features 58, routes 10, components 3, test 3, api 2 · 799 tests	`account.test.tsx`, `admin-listing-edit.test.tsx`	every PR (frontend-check)
End-to-end	Playwright, Chromium	7 specs, 18 test declarations	`buyer-journey.spec.ts`, `quote.spec.ts`	PR (e2e-preview), after deploy (e2e-dev)
Performance	Node workload; k6 (SWE5001)	3 scenarios; 5 k6 tooling test files	`backend/perf/workload.mjs`, `perf/k6/`	by hand (tooling tested in CI)

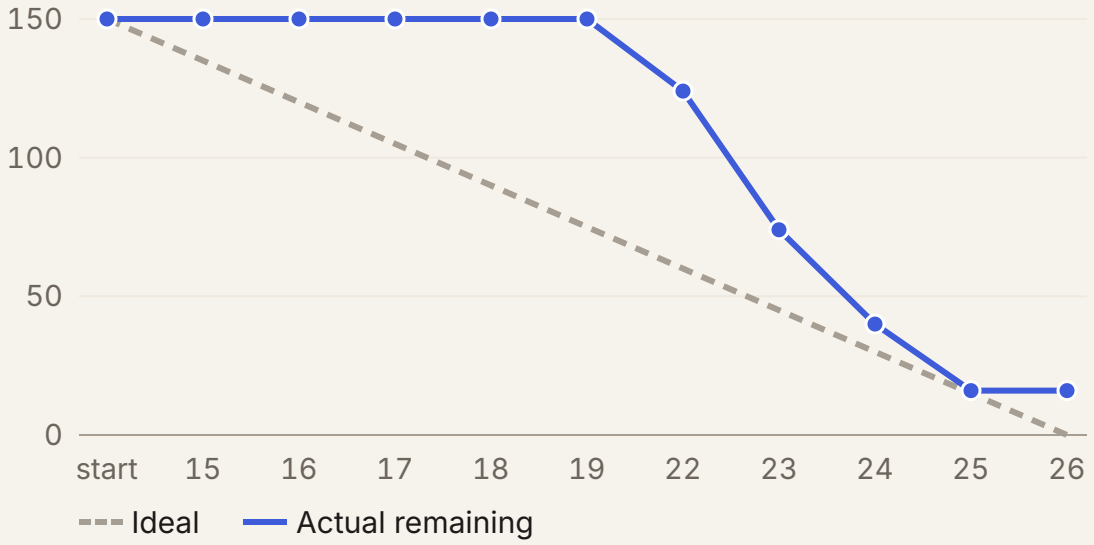
Counted 19 Oct at 8bcb38dd: 183 `*Test.java` (182 + ArchitectureTest), 49 `*IT.java` (45 start containers). Last full runs: backend 1,931 (16 Oct log), frontend 799 (19 Oct log).

JC Lim

Sprint 2 and 3 burndowns

Sprint 2 · remaining hours, 15–26 Sep

150 h planned · 16 h left · reconstructed from merge dates

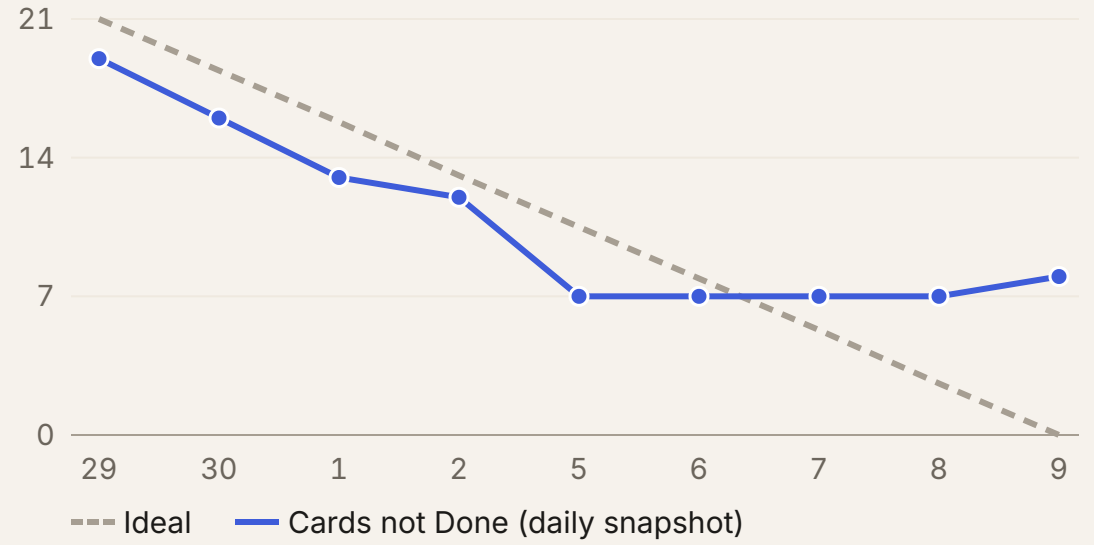


Flat for a week, then a cliff: work merged late in very large pull requests. Retro answer: PRs under 1,500 lines and hours on the board daily.

Source: docs/sprint2-progress-report.typ, "Daily burndown data"

Sprint 3 · open sprint cards, 29 Sep–9 Oct

21 cards · 150 h planned · 33 h left at freeze (review)



Steadier burn, then stuck at 7 from 5 Oct: #223, #224 and #229 waited for Checkout on dev, #226 for the NFR-37 decision. Hours come from the review.

Source: docs/agile/metrics/.csv, sprint-03/review.md*

PS Pan

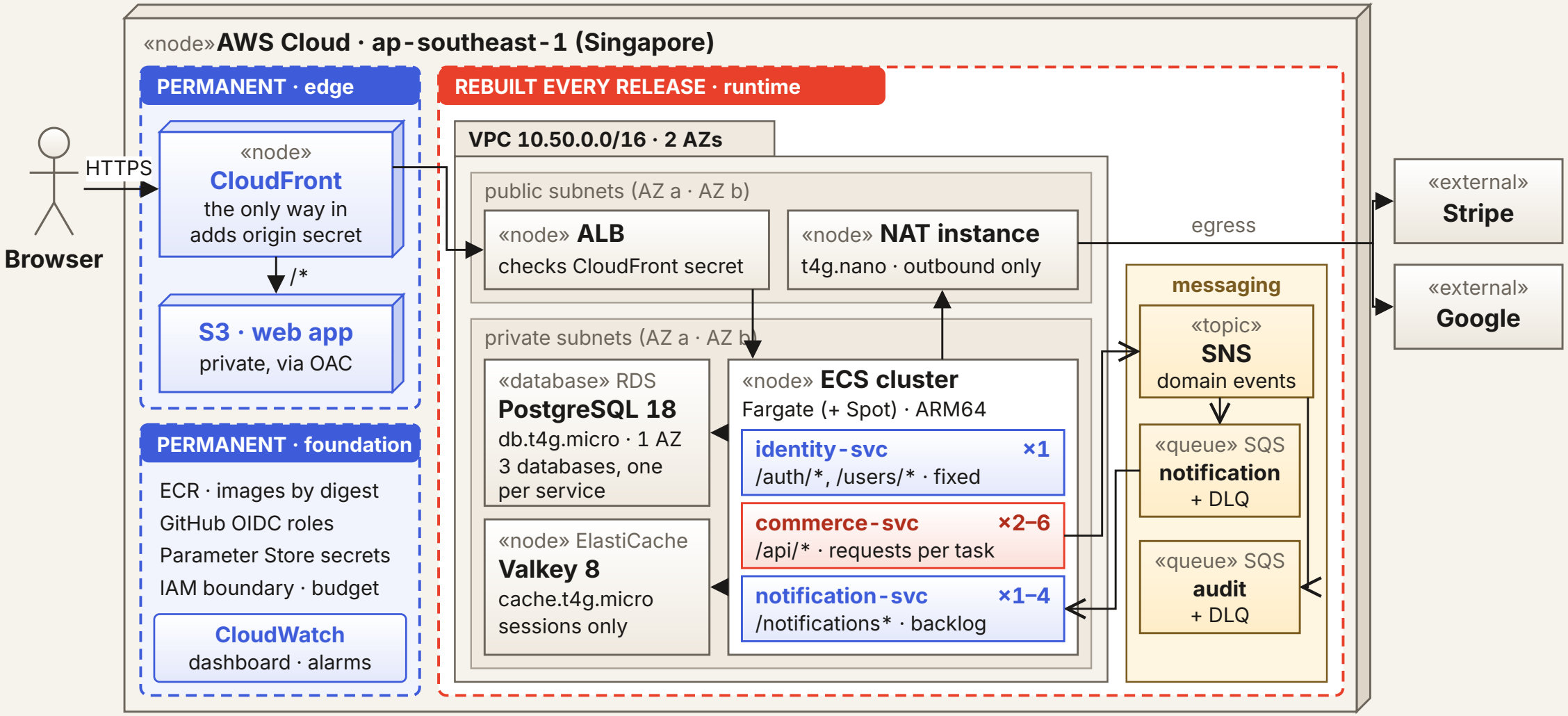
Workflows, required checks and timings

Workflow · trigger	Jobs (required in blue)	Median	Green
ci-backend · PR	**required**: build-quality (13.1 min), docker (2.8 min), SonarCloud Backend; contract-check (12 s)	16.1 min to docker end (18 runs, 16–19 Oct)	18 of 20 finished runs; 16 cancelled by a newer push
ci-backend · push to dev	package-dev 46 s → publish-ecr-dev 73 s → deploy-existing-ec2 40 s; SonarCloud Backend refreshes the dashboard	3.1 min to deployed (88 runs)	deploy 88 of 97; all 9 failures 14–17 Sep (SSM: instance not in a valid state)
ci-frontend · PR	**required**: frontend-check (3.4 min), SonarCloud Frontend; e2e-preview (69 s), intent-stale	4.8 min to e2e-preview end (19 runs)	19 of 20 finished runs
ci-frontend · push to dev	package-dev 35 s → deploy-s3-dev 21 s → e2e-dev 82 s; SonarCloud Frontend	2.7 min to e2e-dev green (94 runs)	deploy 97 of 97; e2e-dev 94 of 97 (3–4 Oct)
security · PR, dev, weekly	**required**: gitleaks (10 s); trivy-deps (72 s), trivy-config, workflow-lint, deployment-bundle when their paths change	12 s on most PRs	PR 97 of 98 finished; dev push 100 of 100
rollback-backend · manual	validate, then SSM rollback to the recorded previous digest or a given one	—	never run
SWE5001 · prod branch	swe5001-release-gate, swe5001-deploy, infra-swe5001 (promotion PR; see the SWE5001 deck)	—	—

Last 100 runs per workflow and event, read with `gh run list` and the jobs API on 19 Oct. PR durations count only runs where the gated jobs ran (docs-only PRs skip them in about 10 s). Push medians run from the push run start, not the merge click.

PS Pan

Production topology (SWE5001)



SWE5001 prod: the same code on ECS, rebuilt from scratch on every release and destroyed after it. SWE5006 dev is slide 18.

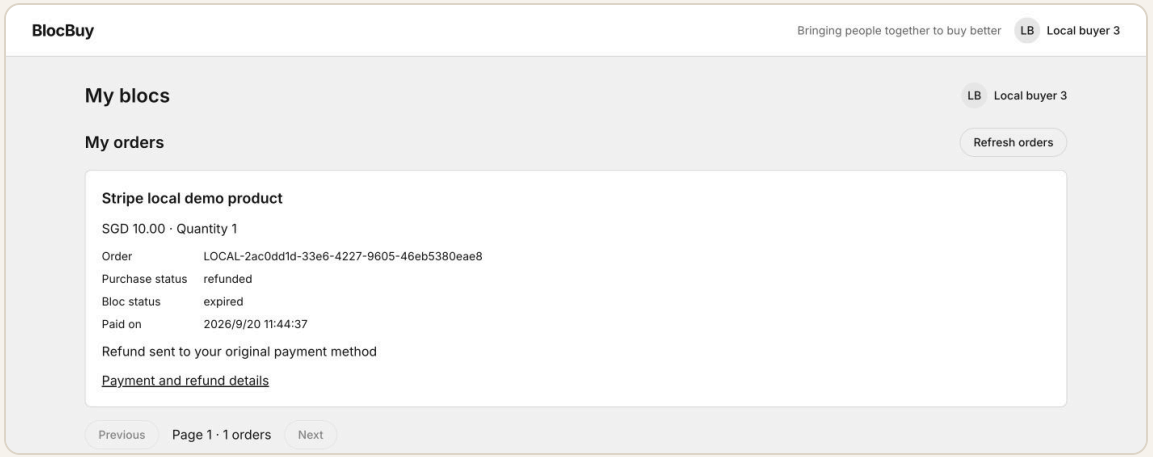
PS Pan

Known gaps and limitations

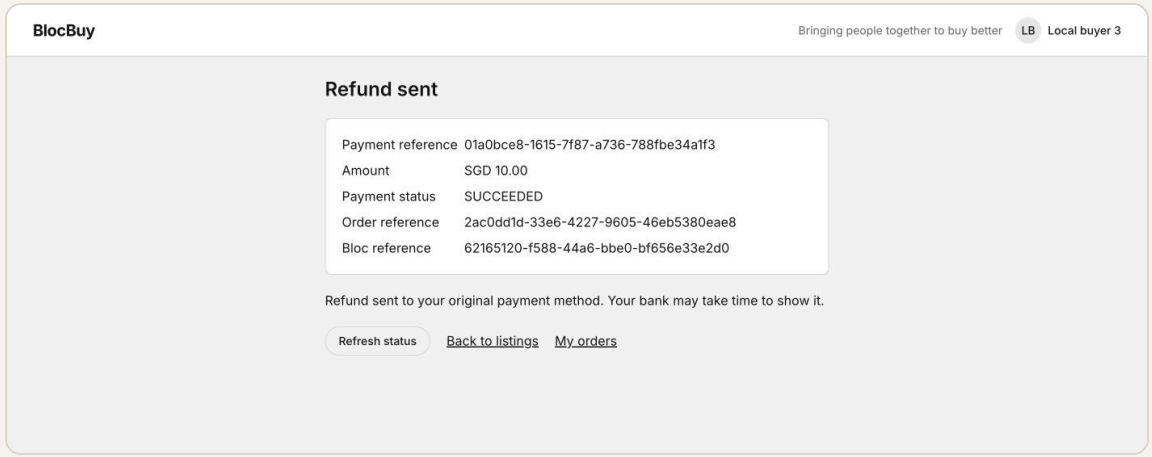
Gap	Effect	Status
CodeQL off	no CodeQL SAST; SonarCloud and SpotBugs remain	ENABLE_CODEQL=false; code scanning not enabled on the repository
CloudFront → ALB over HTTP	/api payloads and the origin header cross the AWS network unencrypted	NFR-37 unmet, #227 open; needs an origin hostname and ACM
SSH open to the internet	port 22 from 0.0.0.0/0 on the dev host, which has a public IP; SSM sessions make it unnecessary	AWS read-only check, 19 Oct; open item 16
Dev EBS volume not encrypted	database and host data at rest unencrypted	AWS read-only check, 19 Oct; not raised as an issue yet
Rollback never exercised	recovery path untested on the real host	rollback-backend.yml: 0 runs; no automatic rollback has fired
No dev alarms	failures are noticed by e2e-dev or by people	all six CloudWatch alarms belong to SWE5001
Dev not zero-downtime	a deploy restarts the only API container (502 on 3–4 Oct)	accepted for dev
Secrets in a host file	manual creation and rotation, root-only 0600	accepted for dev; prod uses Parameter Store
Rebuilt after merge	the deployed image and dist/ are not the PR-scanned artefacts; Trivy not re-run	strict up-to-date checks; ECR scans on push; ADR 0004 text to reconcile
Gitleaks scans selected commits	24 historical matches still to decide	docs/quality/quality-gates.md, 14 Oct

YQ Yorik

Stripe Sandbox refund, local evidence (20 Sep)



My orders: the expired bloc's order is refunded (UI of 20 Sep, since redesigned)



Receipt: "Refund sent", payment 01a0bce8..., SGD 10.00

Scenario	Result on 20 Sep (Team8 Stripe Sandbox, livemode=false)
START and JOIN	two SGD 10 payments → one member and one order each; the bloc reached succeeded, 2 of 2
Webhook replay	replayed valid events changed nothing; a modified payload with the old signature got 400
Expiry	an under-filled bloc expired → one SGD 10 refund obligation; its unpaid Checkout session expired
Refund	one full Sandbox refund `re_3UHbeV5iEVLN2uQJ1V71krwQ`, succeeded; a recheck created no duplicate

Local, not dev: real Stripe Sandbox calls from a local backend and PostgreSQL; no Google sign-in, no deployed environment. On dev the refund waits for #223 and #517. Source: `backend/docs/payment/stripe-sandbox-acceptance-20260920.md`

WZ Eric

Scale-out and load results (SWE5001 prod + dev baseline)

SWE5001 PROD · ECS FARGATE

Scale-out

- Target tracking on ALB requests per target: 12,000 a minute, scale-out cooldown 60 s
- commerce 2–6 tasks in two AZs, notification 1–4 (read back 16 Oct)
- V3 scale-out under k6:

  `[FILL: max commerce tasks, p95, error rate from the 19 Oct session]`
- V4 queue drain:

  `[FILL: backlog peak, notification tasks, drain time (19 Oct)]`

SWE5001 V2 · ECS, 16 OCT

Load, one task

- k6, ten one-minute plateaus from 10 to 200 requests a second, through CloudFront
- 57,600 requests, **0 errors**, run twice
- p95 between 23 and 57 ms on every plateau (target under 500 ms)
- No saturation reached: 200 a second is a tested lower bound, not a limit

DEV CODE · LOCAL RUN, 30 SEP

Workload baseline

100 users each	p95	Errors
Browse (list + detail)	24 ms	0
JOIN, one hot team	57 ms	0
My Orders	18 ms	0

30 paid teams expired within 58 s of their deadlines under that load. One laptop with real HTTP, PostgreSQL and Redis; **not a deployed result**.

Dev, where the demo runs: one EC2 host by design (backend ADR 0008), so no auto-scaling there. Same code, two topologies: scale-out is shown on SWE5001 prod.

Sources: docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md (V2, scaling policy); docs/perf/sprint3-baseline.md (local workload).